

Lecture 3-2 - Recursion

In this lecture, we shall use functions. Unlike methods, functions do not modify the state; rather, they consist of parametrized expressions that denote the value computed from given arguments, e.g.

```
function BinomialSquare(a: real, b: real): real
  ensures BinomialSquare(a, b) == (a + b) * (a + b)
{
  a * a + 2.0 * a * b + b * b
}
```

Functions and methods may call themselves, that is, they may be recursive; this exposes them to the risk of circular definitions or infinite sequences of recursive calls (the next examples do not check in Dafny):

```
method BadDouble(x: int) returns (d: int)
  ensures d == 2 * x
{
  var y := BadDouble(x - 1); // unbounded recursion
  d := y + 2;
}

function Dubious(): nat
{
  1 + Dubious() // unbounded recursion
}
```

However, it is not necessarily the case that a recursive function is undefined, for example:

```
function Fib(n: nat): nat
  decreases n // unnecessary as it is automatic in this case
{
  if n < 2 then n else Fib(n - 2) + Fib(n - 1)
}
```

Dafny checks that a recursive function/method is well-defined by proving that it always terminates (no infinite recursive calls occur). To do this, we use a decreases clause like:

```
decreases n
```

This is a decoration expressing a "termination metric"; the proof consists of checking that the

parameter of Fib in the calls Fib($n - 2$) and Fib($n - 1$) is strictly smaller than the callee's decoration n of Fib(n). Since $n > n - 2$ and $n > n - 1$, and the order on natural numbers is well-founded, namely, no infinite (strictly) descending chain of naturals w.r.t. $>$ exists, this proves that Fib is terminating.

The proof is:

call	proof obligation

Fib(n) calls Fib($n - 2$)	$n > n - 2$
Fib(n) calls Fib($n - 1$)	$n > n - 1$

The order relation depends on the type (in this case, nat). Also, the termination metrics are not unique and may be any (meaningful) expression. For example, another decoration for Fib is decreasing $3 * n + 2$.

```
function Fib'(n: nat): nat
  decreases 3 * n + 2
{
  if n < 2 then n else Fib(n - 2) + Fib(n - 1)
}
```

A more complex example using sequences (a kind of immutable arrays) is:

```
function SeqSum(s: seq<int>, lo: int, hi: int): int
  requires 0 <= lo <= hi <= |s|
  decreases hi - lo
{
  if lo == hi then 0 else s[lo] + SeqSum(s, lo + 1, hi)
}
```

Remark: $s: \text{seq}$ is a sequence of $|s|$ integers $s[i]$ for i in $0..|s| - 1$

The following are exercises 3.0 to 3.3 from Leino's book:

```

// Ex. 3.0
function F(x: int): int
  decreases x - 10 // automatic
{
  if x < 10 then x else F(x - 1)
}

// Ex. 3.1
function G(x: int): int
  decreases x // automatic
{
  if 0 <= x then G(x - 2) else x
}

// Ex. 3.2
function H(x: int): int
  decreases x + 60 // necessary!
{
  if x < -60 then x else H(x - 1)
}

// Ex. 3.3
function I(x: nat, y: nat): int
  decreases x + y // automatic
{
  if x == 0 || y == 0 then 12
  else if x % 2 == y % 2 then I(x - 1, y)
  else I(x, y - 1)
}

```

Lexicographic order

It may be necessary to consider a tuple of expressions in the declaration clause rather than a single expression. Tuples are compared lexicographically:

```

m_1, ... m_h > n_1, ... n_k    if

either h = 0 (the tuple m_1, ... m_h = <> is empty)
or    m_1 > n_1
or    m_1 = n_1 and m_2, ... m_h > n_2, ... n_k

Examples:

2, 5 > 1, 7      as 2 > 1
2, 5 > 2, 4      as 2 = 2 but 5 > 4
2 > 2, 5         as 2 = 2 and <> > 5 (2 is shorter than 2, 5)

```

The following is exercise 3.9 from Leino's book:

```
// Ex. 3.9
function N(x: int, y: int, b: bool): int
  decreases x, b // needed
{
  if x <= 0 || y <= 0 then x + y
  else if b then N(x, y + 3, !b)
  else N(x - 1, y, true)
}
```

Remark: the order over bool is true > false.

Example: Remaining school work (Leino 3.3)

To graduate, you must complete courses from 0 to n. The method `RequiredStudyTime(c)` returns how many hours are still needed to complete the course c:

```
method RequiredStudyTime(c: nat) returns (hours: nat)
  // left unspecified
```

Next, the method `Study(n, h)` models working h hours in course n; it stops when `h == 0` and `n == 0` (the last course). Here, n also represents the number of courses remaining after completing the current one.

```
method Study(n: nat, h: nat)
  decreases n, h // automatic
{
  if h != 0 {
    // if there are still hours to study,
    // study one hour, then proceed
    Study(n, h - 1);
  }
  else if n == 0 {
    assert h == 0;
    // unnecessary, this is just a check for us
    // finished course 0, graduation time
  }
  else {
    // find how much studying is needed for the next course
    var hours := RequiredStudyTime(n - 1);
    // get started with course n - 1
    Study(n - 1, hours);
  }
}
```

Mutually recursive methods

Methods or functions are mutually recursive if they call each other. In the case of mutually recursive methods, proving termination is more difficult, as the proof may depend on more than one

parameter, and the number of parameters may differ from one method to another. Here, lexicographic ordering is helpful.

In the more elaborate example that follows, n represents the number of completed courses out of 40:

```
method StudyPlan(n: nat)
  requires n <= 40 // n completed courses out of 40
  decreases 40 - n
{
  if n == 40 {
    // done
  } else {
    var hours := RequiredStudyTime(n);
    Learn(n, hours);
  }
}
```

The method `StudyPlan` calls `Learn`, which is similar to the `Study` of the previous example. It calls `StudyPlan` when no hours of work remain for course n , resulting in mutual recursion:

```
method Learn(n: nat, h: nat)
  requires n < 40
  decreases 40 - n, h
{
  if h == 0 {
    // done with class n, continue with
    // the rest of the study plan
    StudyPlan(n + 1);
  } else {
    // some learning to take place here
    Learn(n, h - 1);
  }
}
```

The proof is:

call	proof obligation

StudyPlan calls Learn	$40 - n > 40 - n, h$
Learn calls StudyPlan	$40 - n, h > 40 - (n + 1)$
Learn calls Learn	$40 - n, h > 40 - n, h - 1$

References: see Leino, chapter 3.